

ALU Verification

using Cocotb in Python

Verification Report

By Ahmed H. Ashour

Cocotb Testbench

The testbench drives random values into the ALU for both arithmetic (add/subtract) and logical (AND/XOR) operations using timers since there is no clock signal. A background checker continuously compares the DUTs output against expected results calculating pass/fail counters in the end the checker is stopped and as a summary it prints the total number of passed and failed tests.

Drive arithmetic coroutine

This coroutine initializes the ALU inputs then drives random values into a and b while changing the opcode between addition and subtraction each operation is separated by a short timer delay "2ns".

```
testbench.py

async def drive_arithmetic(dut):
    dut.a.value = 0
    dut.b.value = 0
    dut.opcode.value = 0
    await Timer(2, units='ns')
    for i in range(10):
        dut.a.value = random.randint(0, 15)
        dut.b.value = random.randint(0, 15)
        dut.opcode.value = 0 # addition operation
        await Timer(2, units='ns')
        dut.opcode.value = 1 # subtraction operation
        await Timer(2, units='ns')
```

Figure 1: drive arithmetic coroutine

Drive logical coroutine

This coroutine initializes the ALU inputs then drives random values into a and b while changing the opcode between AND and XOR each operation is separated by a short timer delay "2ns".

```
testbench.py

async def drive_logical(dut):
    dut.a.value = 0
    dut.b.value = 0
    dut.opcode.value = 0
    await Timer(2, units='ns')
    for i in range(10):
        dut.a.value = random.randint(0, 15)
        dut.b.value = random.randint(0, 15)
        dut.opcode.value = 2 # AND operation
        await Timer(2, units='ns')
        dut.opcode.value = 3 # XOR operation
        await Timer(2, units='ns')
```

Figure 2 : drive logical coroutine

Checker coroutine

The checker coroutine samples the DUT outputs every 2 ns it and compares them against the expected arithmetic or logical result based on the opcode. It updates the global pass/fail counters and prints if each transaction matched or mismatched the expected results.

```
testbench.py

async def check_results(dut):
    #to update the counters
    global pass_count, fail_count
    # check results every 2 ns
    while True:
        await Timer(2, units='ns')
        await ReadOnly()

        if dut.opcode.value == 0: # addition operation
            expected_result = dut.a.value + dut.b.value
        elif dut.opcode.value == 1: # subtraction operation
            expected_result = dut.a.value - dut.b.value
        elif dut.opcode.value == 2: # AND operation
            expected_result = dut.a.value & dut.b.value
        elif dut.opcode.value == 3: # XOR operation
            expected_result = dut.a.value ^ dut.b.value
        else:
            expected_result = 0

        if dut.result.value == expected_result:
            pass_count += 1
            cocotb.log.info("test passed for opcode %d ,a: %d, b: %d ,result: %d", dut.opcode.value,
                dut.a.value, dut.b.value, dut.result.value)
        else:
            fail_count += 1
            cocotb.log.info("test failed for opcode %d, a: %d, b: %d, result: %d, expected: %d",
                dut.opcode.value, dut.a.value, dut.b.value, dut.result.value, expected_result)
```

Figure 3 : Checker Coroutine

Cocotb Test

This cocotb test runs the **checker** in the background then runs the **arithmetic driver** and **logical driver** to feed randomized stimulus into the ALU and when they finish it stops the checker and prints a summary showing how many tests passed and failed.

```
testbench.py

@cocotb.test()
async def alu_tb(dut):
    # start the checker in the background
    checker = cocotb.start_soon(check_results(dut))
    #run arithmetic driver
    await drive_arithmetic(dut)
    #run logical driver
    await drive_logical(dut)
    #stop the checker
    checker.kill()
    #results
    cocotb.log.info("tests passed = %d", pass_count)
    cocotb.log.info("tests failed = %d", fail_count)
```

Figure 4 : Cocotb Test

The Makefile & run.bash file

The Makefile configures cocotb to run the ALU design with the Icarus Verilog simulator on EDA playground specifying the source file (alu.sv), the top-level module (alu), and the Python testbench (testbench.py).

```
Makefile

TOPLEVEL_LANG = verilog
VERILOG_SOURCES = $(PWD)/alu.sv
TOPLEVEL = alu
MODULE = testbench
SIM = icarus
include $(shell cocotb-config --makefiles)/Makefile.sim
```

Figure 5 : Makefile

The run script sets the simulator environment variable to Icarus Verilog and then “make” which triggers cocotb’s Makefile to compile and run the testbench.

```
run.bash

#!/bin/bash
export SIM=icarus
make
```

Figure 6 : run.bash file

now to run this setup on eda playground I needed to choose the option for using a run.bash file in the run options

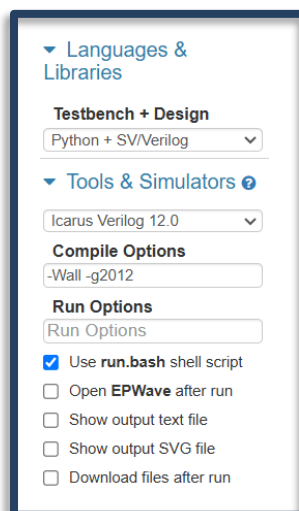
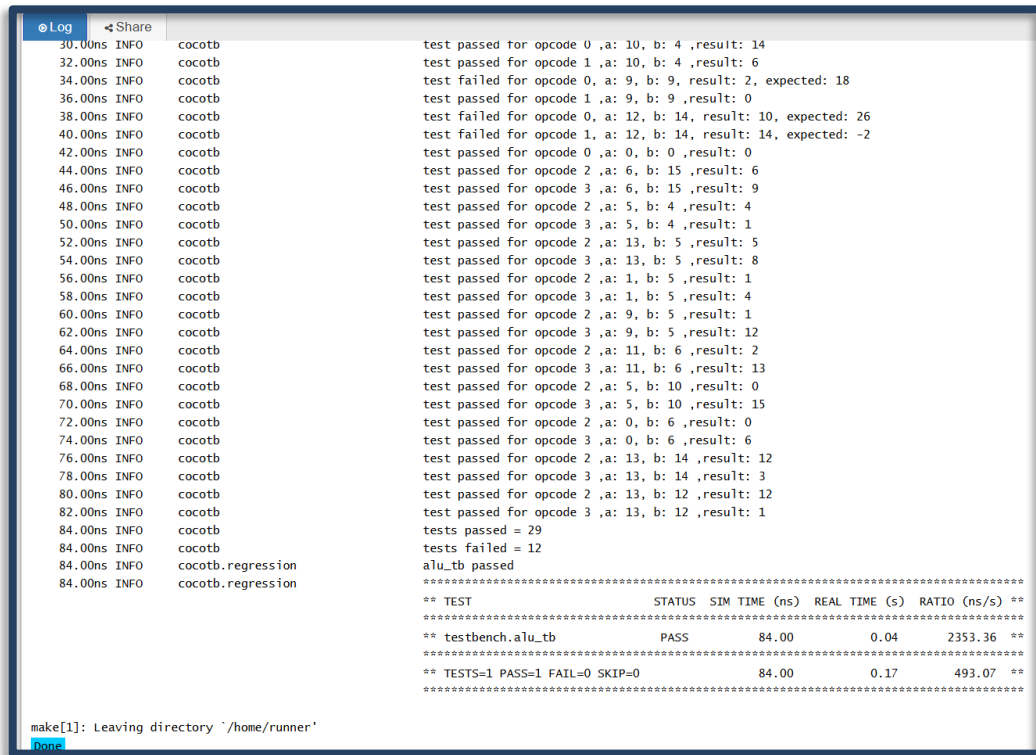


Figure 7 : EDA playground setting

Results

The results show that the alu passed 29 test and failed 12 which are the arithmetic tests that results in an addition overflow or negative subtraction results since the alu doesn't account for them.



```
Log Share
30.00ns INFO cocotb test passed for opcode 0 ,a: 10, b: 4 ,result: 14
32.00ns INFO cocotb test passed for opcode 1 ,a: 10, b: 4 ,result: 6
34.00ns INFO cocotb test failed for opcode 0 ,a: 9, b: 9, result: 2, expected: 18
36.00ns INFO cocotb test passed for opcode 1 ,a: 9, b: 9 ,result: 0
38.00ns INFO cocotb test failed for opcode 0 ,a: 12, b: 14, result: 10, expected: 26
40.00ns INFO cocotb test failed for opcode 1 ,a: 12, b: 14, result: 14, expected: -2
42.00ns INFO cocotb test passed for opcode 0 ,a: 0, b: 0 ,result: 0
44.00ns INFO cocotb test passed for opcode 2 ,a: 6, b: 15 ,result: 6
46.00ns INFO cocotb test passed for opcode 3 ,a: 6, b: 15 ,result: 9
48.00ns INFO cocotb test passed for opcode 2 ,a: 5, b: 4 ,result: 4
50.00ns INFO cocotb test passed for opcode 3 ,a: 5, b: 4 ,result: 1
52.00ns INFO cocotb test passed for opcode 2 ,a: 13, b: 5 ,result: 5
54.00ns INFO cocotb test passed for opcode 3 ,a: 13, b: 5 ,result: 8
56.00ns INFO cocotb test passed for opcode 2 ,a: 1, b: 5 ,result: 1
58.00ns INFO cocotb test passed for opcode 3 ,a: 1, b: 5 ,result: 4
60.00ns INFO cocotb test passed for opcode 2 ,a: 9, b: 5 ,result: 1
62.00ns INFO cocotb test passed for opcode 3 ,a: 9, b: 5 ,result: 12
64.00ns INFO cocotb test passed for opcode 2 ,a: 11, b: 6 ,result: 2
66.00ns INFO cocotb test passed for opcode 3 ,a: 11, b: 6 ,result: 13
68.00ns INFO cocotb test passed for opcode 2 ,a: 5, b: 10 ,result: 0
70.00ns INFO cocotb test passed for opcode 3 ,a: 5, b: 10 ,result: 15
72.00ns INFO cocotb test passed for opcode 2 ,a: 0, b: 6 ,result: 0
74.00ns INFO cocotb test passed for opcode 3 ,a: 0, b: 6 ,result: 6
76.00ns INFO cocotb test passed for opcode 2 ,a: 13, b: 14 ,result: 12
78.00ns INFO cocotb test passed for opcode 3 ,a: 13, b: 14 ,result: 3
80.00ns INFO cocotb test passed for opcode 2 ,a: 13, b: 12 ,result: 12
82.00ns INFO cocotb test passed for opcode 3 ,a: 13, b: 12 ,result: 1
84.00ns INFO cocotb tests passed = 29
84.00ns INFO cocotb tests failed = 12
84.00ns INFO cocotb.alu_tb passed
84.00ns INFO cocotb.regression
84.00ns INFO cocotb.regression

*****
** TEST STATUS SIM TIME (ns) REAL TIME (s) RATIO (ns/s) **
*****
** testbench.alu_tb PASS 84.00 0.04 2353.36 **
*****
** TESTS=1 PASS=1 FAIL=0 SKIP=0 84.00 0.17 493.07 **
*****

make[1]: Leaving directory `/home/runner'
Done
```

Figure 8 : EDA Playground log window